Commit hack fix

jak pipeline jutra wymusza bezpieczeństwo

Tomasz Świerszcz



LEAN ARCHITECTURE FRAMEWORK

https://www.linkedin.com/in/tswierszcz/

tswierszcz@lafinstitute.org

Security is Architecture

Proces wytwórczy jutra: SDLC, guardrails i pipeline, który egzekwuje architekturę — nie tylko buduje artefakty.

01 **Release day**
feature gotowy, security na końcu

02 **Zwrot**
security = właściwość systemu

03 **Dowód**
live demo fail → fix → pass

04 **Architektura**
trust boundaries, policies, guardrails

05 **Skala**
enterprise patterns i golden paths

18:42. Release day.

Feature jest gotowy. Testy są zielone. Biznes czeka. I wtedy pojawia się klasyczne: „security jeszcze nie sprawdziło”.

- kosztowna poprawka zamiast szybkiego feedbacku
- frustracja: dev vs security
- release staje się negocjacją, nie procesem



Prawdziwy problem nie brzmi: „czy mamy security?”
Brzmi: „gdzie w systemie egzekwujemy zasady?”

“Security nie jest etapem SDLC. Security jest właściwością architektury.”

Jeżeli zasada bezpieczeństwa istnieje tylko w dokumencie lub w głowie eksperta, to system jej nie egzekwuje.

Dokument → rekomendacja · Pipeline → egzekucja

Od diagramu do egzekwowalnej architektury

Trzy rzeczy, które architekt projektuje w SDLC jutra

Trust boundaries

Gdzie kończy się zaufanie i zaczyna ryzyko? Każde przejście między komponentami to kandydat na kontrolę.

01

Threat model

STRIDE daje język do rozmowy o zagrożeniach na etapie designu, zanim powstanie podatny kod.

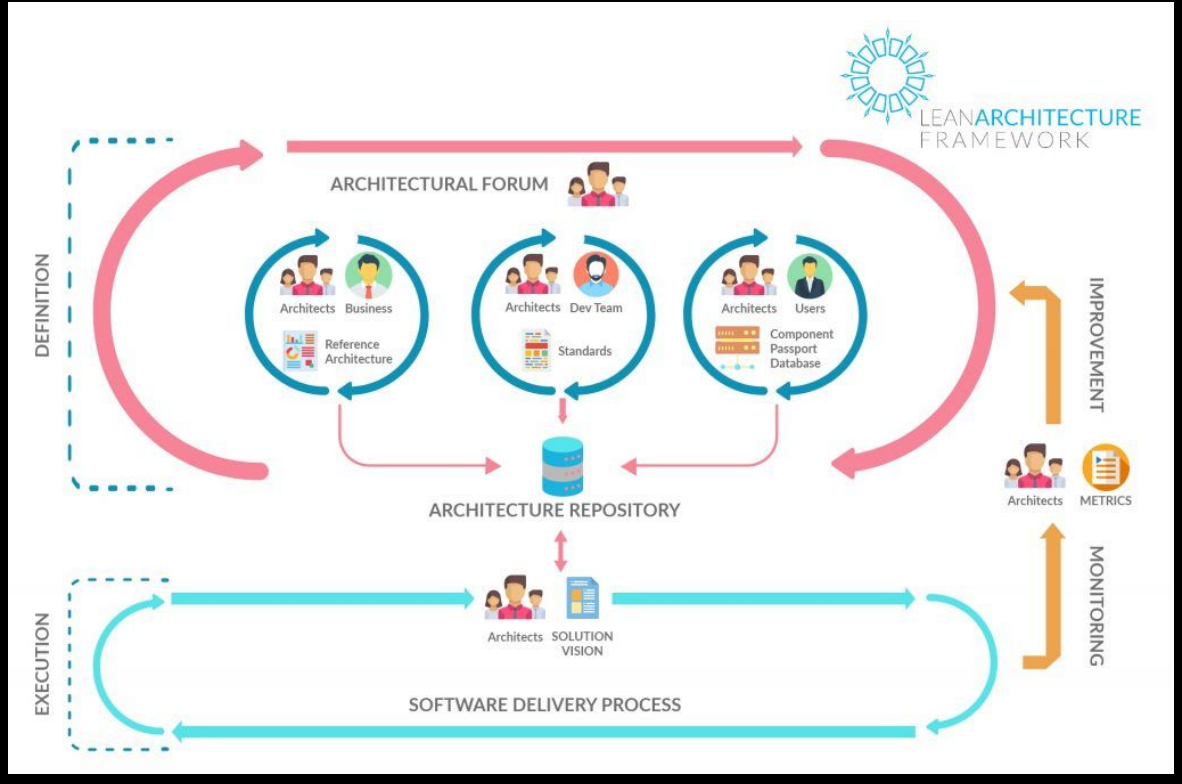
02

Security controls

AuthN, AuthZ, walidacja wejścia, szyfrowanie, sekrety, audyt — ale jako kontrolki w systemie, nie tylko zalecenia.

03

Architecture Governance Proces



Pipeline = enforcement layer architektury

Nowy SDLC nie prosi o zgodę. On sprawdza, czy system spełnia reguły.



Guardrails > gates

Gate to człowiek blokujący człowieka. Guardrail to system, który prowadzi developera bez dyskusji o wyjątkach.

Demo · Krok 1 — podatny kod

app/UserController.java

```java
public class UserController {
    public ResultSet getUser(Connection connection,
String userId) throws Exception {
        String query = "SELECT * FROM users WHERE id = "
+ userId;
        Statement stmt = connection.createStatement();
        return stmt.executeQuery(query);
    }
}
```

- „to jest zwykła, banalna konkatenacja SQL”
- „feature działa — ale kod jest niebezpieczny”
- „i właśnie dlatego chcemy feedback zanim dojdziemy do produkcji”

Demo · Krok 2 — uruchom pipeline

Jedna komenda. Tak ma wyglądać proces, któremu można ufać.

terminal

```
$ cd demo-secure-sdlc
$ ./pipeline.sh

===> [1/3] Build
Build OK
===> [2/3] SAST - Semgrep
```

- nie ma osobnego „security review call”
- kontrola jest częścią zwykłego pipeline
- feedback wraca natychmiast do autora zmian

Demo · Krok 3 — Semgrep zatrzymuje release

To jest moment, w którym pipeline mówi „nie”.

terminal · expected output

```
===> [2/3] SAST - Semgrep

app/UserController.java
   ❱❱❱ java.sql-injection
         Possible SQL injection via string
concatenation
         3¦ String query = "SELECT * FROM users
WHERE id=" + userId;

✖ 1 finding (pipeline stopped)
```

„Nie mamy tutaj security teamu. Mamy pipeline, który egzekwuje zasady.”

Demo · Krok 4 — poprawka bez heroics

Nie kodujemy długo. Guardrail prowadzi do bezpiecznej ścieżki.

KROK 4 · fix

terminal

```
$ cp
app_fixed/UserController.java
app/UserController.java
$ git diff --
app/UserController.java
```

app_fixed/UserController.java

```
1 public class UserController {
   public ResultSet getUser(Connection connection,
String userId) throws Exception {
       String query = "SELECT * FROM users WHERE id
= ?";
       PreparedStatement stmt =
connection.prepareStatement(query);
       stmt.setString(1, userId);
       return stmt.executeQuery();
}
```

Dlaczego to jest ważne

- developer dostaje jasny kierunek
- poprawka jest lokalna i szybka
- proces promuje właściwy wzorzec

To jest guardrail: nie tylko blokuje — prowadzi.

Demo · Krok 5 — pipeline przechodzi

Ten sam proces. Inny wynik. Bo artefakt przestał łamać regułę.

terminal · expected output

```
$ ./pipeline.sh

===> [1/3] Build
Build OK
===> [2/3] SAST - Semgrep
No findings.
===> [3/3] Policy check - Conftest
0 tests failed

===> Pipeline OK
```

To nie jest happy end.

**To jest dowód,
że proces stał się
przewidywalny.**

Mniej negocjacji.
Więcej egzekucji reguł.

Policy as Code – Conftest

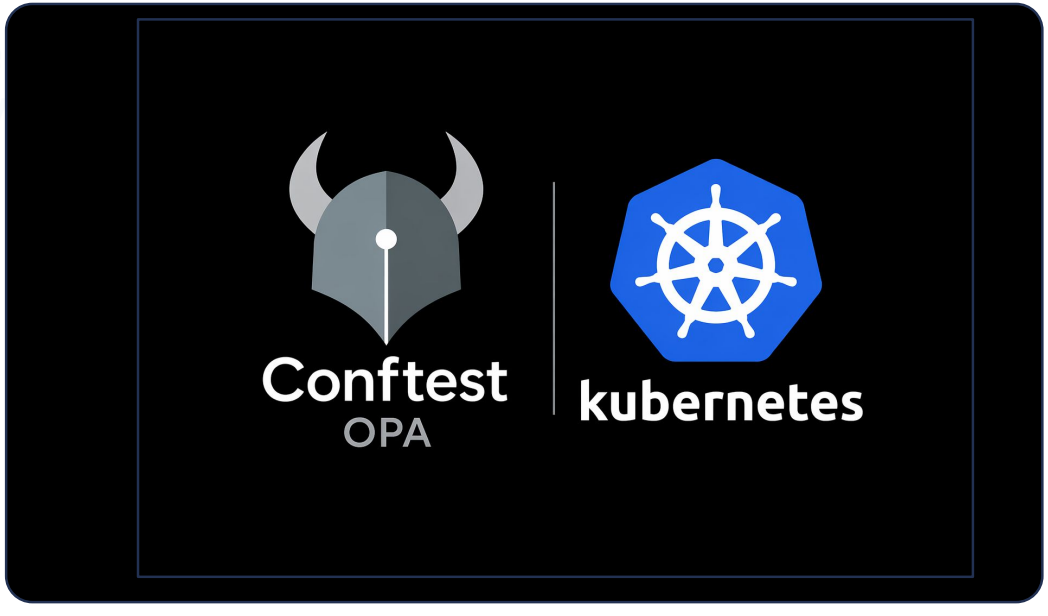
Pipeline egzekwuje architekturę – nie człowiek

Conftest w pipeline

✔ waliduje konfigurację (YAML / JSON)
✔ egzekwuje zasady architektoniczne
✔ blokuje deployment przy naruszeniach

01

02



Trzy architektoniczne wnioski z tego demo

To nie było demo narzędzi. To było demo odpowiedzialności systemu.

Feedback przesunął się w lewo

Problem wrócił do autora kodu w tym samym flow pracy.

01

Reguła stała się wykonywalna

Security przestało być opinią. Stało się wynikiem kontroli.

02

Architektura dostała mechanizm egzekucji

Pipeline potwierdza, że ustalone zasady faktycznie obowiązują.

03

Jak to urosnąć z jednego repo do enterprise

To nie jest „one-off demo”. To jest wzorzec operacyjny.

Mechanizmy

- threat model jako element design review
- repo templates i golden paths
- wspólne reguły Semgrep / secrets / IaC
- policy bundles dla platformy
- metryki: fail rate, MTTR, exception debt

Zmiana roli architekta

z „reviewera diagramów”
→ do „autora guardrails”
→ do „projektanta platform constraints”

Najlepszy architekt jutra nie rysuje najładniejszych diagramów.
Projektuje zasady, których system nie potrafi złamać.

Co dalej: self-healing pipelines i AI-assisted fixes

Następny krok to proces, który nie tylko wykrywa — ale i proponuje bezpieczną poprawkę.



Ale uwaga: AI nie zastępuje architektury.
AI wzmacnia guardrails, które architekt już zdefiniował.

Security nie jest decyzją człowieka.

Security jest wynikiem systemu.

Jeżeli proces wytwórczy jutra ma być szybszy, bezpieczniejszy i mniej konfliktowy,
to architektura musi być egzekwowalna — a pipeline musi umieć powiedzieć: pass albo fail.

Dziękuję

Q&A

Pytania i odpowiedzi